

UNIVERSIDAD DE GUADALAJARA
Centro Universitario de Ciencias Exactas e Ingenierías



MATERIA: Análisis de Algoritmos

Clave: IL355

Sección: D01

Calendario: 2026 A

Sprint 1

Integrantes:

Nayeli Ximena Luna Gallegos
Axel Guillermo Martinez Martinez
Juan Carlos Mijares Ahuatzi

Códigos:

220442484
220450053
2211344872

SPRINT 1 — Módulo Base y Detección Inicial

1.1 Introducción

En este sprint se establecieron los fundamentos del sistema: la detección de vehículos en video mediante YOLOv8, la visualización de bounding boxes y la simulación de datos vehiculares para el módulo de conteo y análisis de métricas simples.

1.2 Actividades Realizadas

Actividad	Responsable	Descripción	Entregable
Ejecutar YOLO en video	Juan	Cargar el modelo YOLO y detectar vehículos en un video de prueba	Script YOLO + video con detecciones
Mostrar detecciones en pantalla	Juan	Dibujar bounding boxes sobre los vehículos detectados	Código + capturas de pantalla
Simular datos de vehículos	Ximena	Generar listas de vehículos con coordenadas (x, y)	Script + dataset en listas/dicts
Organizar vehículos por frame	Ximena	Estructurar datos simulados en frames para análisis	Diccionario por frames
Procesar vehículos simulados	Axel	Transformar coordenadas en métricas: densidad, flujo, dispersión	Script con cálculo de métricas
Medir tiempos de ejecución	Axel	Evaluar el tiempo de procesamiento al aumentar vehículos simulados	Tabla de tiempos + gráfica

SPRINT 1 — Resultados por Integrante

Juan Carlos Mijares Ahuatzi — Detección YOLO en Video

Actividad 1: Ejecutar YOLO en Video

La ejecución de YOLO en video consiste en analizar un archivo multimedia cuadro por cuadro (frame a frame) para encontrar objetos en movimiento de forma continua. Este proceso arranca utilizando la librería OpenCV para abrir el archivo trafico.mp4 y leerlo mediante un ciclo while. En cada iteración, el programa toma una imagen individual del video y se la pasa a la inteligencia artificial (YOLOv8), pidiendo que escanee la imagen y filtre exclusivamente vehículos. Acto seguido, el sistema toma los resultados obtenidos, extrae las posiciones exactas para imprimirlas en la consola, y dibuja las cajas delimitadoras sobre la imagen original. Finalmente, esta secuencia de imágenes ya procesadas se proyecta una tras otra en una ventana de tu escritorio, creando la visualización de un video analizado en tiempo real, operando con los umbrales de confianza (0.25) y limpieza (IoU 0.7) que el modelo trae de fábrica.

Código

Python

```
import cv2
from ultralytics import YOLO

model = YOLO('yolov8n.pt')

video_path = 'trafico.mp4'
cap = cv2.VideoCapture(video_path)

while cap.isOpened():
    exito, frame = cap.read()

    if exito:
        resultados = model(frame, classes=[2, 3, 5, 7])

        frame_annotado = resultados[0].plot()

        cv2.imshow("Detección de Vehículos con YOLOv8", frame_annotado)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    else:
        break

cap.release()
cv2.destroyAllWindows()
```

Evidencia de Ejecución

```

0: 384x640 7 cars, 2 buss, 2 trucks, 57.0ms
Speed: 33.1ms preprocess, 57.0ms inference, 1.2ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 8 cars, 2 buss, 2 trucks, 33.8ms
Speed: 1.9ms preprocess, 33.8ms inference, 0.7ms postprocess per image at shape (1, 3, 384, 640)

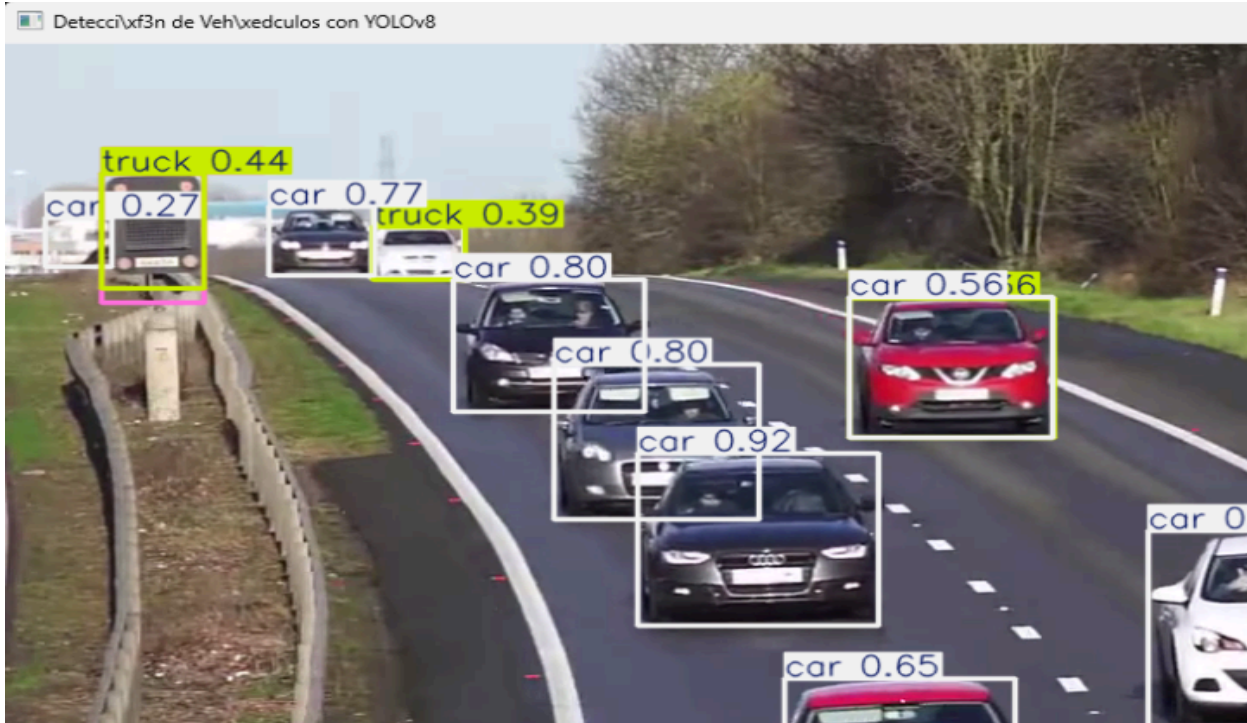
0: 384x640 8 cars, 2 buss, 1 truck, 33.7ms
Speed: 1.8ms preprocess, 33.7ms inference, 0.6ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 8 cars, 2 buss, 33.1ms
Speed: 1.7ms preprocess, 33.1ms inference, 0.9ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 8 cars, 2 buss, 30.9ms
Speed: 1.2ms preprocess, 30.9ms inference, 0.6ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 8 cars, 2 buss, 29.6ms
Speed: 1.2ms preprocess, 29.6ms inference, 0.7ms postprocess per image at shape (1, 3, 384, 640)

```



Actividad 2: Mostrar Detecciones en Pantalla (Bounding Boxes)

El uso de las bounding boxes (cajas delimitadoras) sirve para enmarcar y localizar visualmente cada vehículo detectado dentro del video en tiempo real. Su implementación está completamente automatizada y simplificada por la librería de YOLO: primero, el modelo analiza la imagen y calcula matemáticamente las coordenadas espaciales ocultas de los objetos detectados; posteriormente, el método `.plot()` toma esos datos y se encarga de dibujar físicamente los rectángulos y las etiquetas de texto directamente sobre la imagen original. Esto facilita enormemente el desarrollo, ya que te permite obtener una visualización clara y profesional sin tener que programar manualmente el trazado de líneas y cuadros utilizando las herramientas de dibujo tradicionales de OpenCV.

Código

Partes específicas que hacen las bounding boxes:

Python

```
#El cálculo matemático (¿Dónde están?)
resultados = model(frame, classes=[2, 3, 5, 7])

#El dibujo visual (Pintar las cajas)
frame_annotado = resultados[0].plot()
```

Conclusión de Juan

En cuanto a la implementación de YOLO en video y el uso de bounding boxes, lo que funcionó muy bien fue la integración directa con OpenCV para analizar el archivo cuadro por cuadro y la enorme facilidad que brinda el método `.plot()` para dibujar automáticamente los recuadros de colores, ahorrándonos la necesidad de programar código gráfico manual. Las dificultades principales se presentaron al intentar comprender y controlar el comportamiento oculto o que hace a nuestras espaldas el modelo al generar estas cajas; lidiar con falsos positivos o recuadros superpuestos sobre un mismo vehículo.

Nayeli Ximena Luna Gallegos — Módulo de Conteo

Actividad 1: Simular Datos de Vehículos

En esta actividad se implementó un script en Python para simular datos de vehículos, generando listas con coordenadas (x, y) que representan su posición. Esto permite crear datos de prueba sin depender de video real, facilitando el desarrollo y validación de las siguientes etapas del sistema.

Código

```
Python
import random

# CONFIGURACIÓN

NUM_FRAMES = 10
VEHICULOS_POR_FRAME = 5

ANCHO = 640
ALTO = 480

# 1. SIMULAR VEHÍCULOS

def generar_vehiculo(id_vehiculo):
    return {
        "id": id_vehiculo,
        "x": random.randint(0, ANCHO),
        "y": random.randint(0, ALTO)
    }

def simular_frame(frame_num, num_vehiculos, id_inicio):
    vehiculos = []
    for i in range(num_vehiculos):
        vehiculo = generar_vehiculo(id_inicio + i)
        vehiculos.append(vehiculo)
    return vehiculos

# 3. EJECUCIÓN

if __name__ == "__main__":
    data = generar_dataset(NUM_FRAMES, VEHICULOS_POR_FRAME)
```

```
# Mostrar ejemplo
for frame, vehiculos in data.items():
    print(f"\n{frame}:")
    for v in vehiculos:
        print(v)
```

Salida del Script (Dataset)

```
frame_0:
{'id': 0, 'x': 286, 'y': 476}
{'id': 1, 'x': 224, 'y': 380}
{'id': 2, 'x': 616, 'y': 133}
{'id': 3, 'x': 498, 'y': 204}
{'id': 4, 'x': 384, 'y': 175}
```

```
frame_1:
{'id': 5, 'x': 86, 'y': 479}
{'id': 6, 'x': 425, 'y': 473}
{'id': 7, 'x': 60, 'y': 109}
{'id': 8, 'x': 245, 'y': 218}
{'id': 9, 'x': 69, 'y': 467}
```

```
frame_2:
{'id': 10, 'x': 102, 'y': 79}
{'id': 11, 'x': 636, 'y': 207}
{'id': 12, 'x': 441, 'y': 396}
{'id': 13, 'x': 75, 'y': 132}
{'id': 14, 'x': 388, 'y': 131}
```

```
frame_3:
{'id': 15, 'x': 80, 'y': 313}
{'id': 16, 'x': 139, 'y': 424}
{'id': 17, 'x': 450, 'y': 353}
{'id': 18, 'x': 183, 'y': 125}
{'id': 19, 'x': 70, 'y': 380}
```

```
frame_4:
{'id': 20, 'x': 332, 'y': 65}
{'id': 21, 'x': 343, 'y': 422}
{'id': 22, 'x': 139, 'y': 476}
{'id': 23, 'x': 398, 'y': 107}
{'id': 24, 'x': 273, 'y': 46}
```

```
frame_5:
{'id': 25, 'x': 512, 'y': 45}
{'id': 26, 'x': 43, 'y': 51}
{'id': 27, 'x': 20, 'y': 97}
{'id': 28, 'x': 226, 'y': 460}
{'id': 29, 'x': 247, 'y': 382}
```

```
frame_6:
{'id': 30, 'x': 330, 'y': 8}
{'id': 31, 'x': 115, 'y': 356}
{'id': 32, 'x': 371, 'y': 118}
{'id': 33, 'x': 415, 'y': 64}
{'id': 34, 'x': 11, 'y': 256}
```

```
frame_7:
{'id': 35, 'x': 217, 'y': 7}
{'id': 36, 'x': 573, 'y': 248}
{'id': 37, 'x': 518, 'y': 451}
{'id': 38, 'x': 235, 'y': 206}
{'id': 39, 'x': 397, 'y': 293}
```

```
frame_8:
{'id': 40, 'x': 494, 'y': 160}
{'id': 41, 'x': 327, 'y': 415}
{'id': 42, 'x': 427, 'y': 338}
{'id': 43, 'x': 173, 'y': 224}
{'id': 44, 'x': 253, 'y': 347}
```

```
frame_9:
{'id': 45, 'x': 396, 'y': 248}
{'id': 46, 'x': 288, 'y': 453}
{'id': 47, 'x': 620, 'y': 102}
{'id': 48, 'x': 258, 'y': 189}
{'id': 49, 'x': 235, 'y': 476}
```

Actividad 2: Organizar Vehículos por Frame

Se implementó una función que organiza los vehículos simulados en distintos frames, creando un diccionario donde cada frame contiene una lista de vehículos con sus coordenadas. Esto permite estructurar los datos de forma similar a un video, facilitando su uso en etapas posteriores como el conteo y análisis.

Código

```
Python
import random

# CONFIGURACIÓN

NUM_FRAMES = 10
VEHICULOS_POR_FRAME = 5

ANCHO = 640
ALTO = 480

# 2. ORGANIZAR POR FRAMES

def generar_dataset(num_frames, vehiculos_por_frame):
    dataset = {}
    id_global = 0

    for frame in range(num_frames):
        vehiculos = simular_frame(frame, vehiculos_por_frame, id_global)
        dataset[f"frame_{frame}"] = vehiculos
        id_global += vehiculos_por_frame

    return dataset

# 3. EJECUCIÓN

if __name__ == "__main__":
    data = generar_dataset(NUM_FRAMES, VEHICULOS_POR_FRAME)

    # Mostrar ejemplo
    for frame, vehiculos in data.items():
        print(f"\n{frame}:")
```



```
for v in vehiculos:  
    print(v)
```

Ejemplo de Estructura de Frame

```
frame_0:  
{'id': 0, 'x': 396, 'y': 427}  
{'id': 1, 'x': 600, 'y': 22}  
{'id': 2, 'x': 113, 'y': 95}  
{'id': 3, 'x': 564, 'y': 103}  
{'id': 4, 'x': 437, 'y': 111}
```

Conclusión de Ximena

En este script logré simular vehículos y organizarlos por frames usando un diccionario, lo que me permitió representar la estructura básica de un video para pruebas. Funcionó correctamente para generar y visualizar los datos, aunque me surgieron dudas en el proceso sobre la creación de muchos vehículos distintos, o sea los id, por frame. Estas dudas me ayudaron a entender que la simulación no es completamente realista, ya que en un video real los mismos vehículos se mantienen entre frames y solo cambian de posición. Finalmente aprendí la importancia de estructurar datos y el cómo representar objetos mediante coordenadas (x, y) como base para el conteo y análisis posteriores.

Axel Guillermo Martinez Martinez — Módulo de Análisis

Actividad 1: Procesar Vehículos Simulados

En este primer sprint se diseñó lo que será el algoritmo base para el procesamiento de frames de video para a partir de ahí empezar a generar métricas de análisis útiles para el proyecto, solamente se pusieron métricas simples como el determinar en qué zona (simulada) esta el vehiculo, la densidad total de vehículos que hay y la dispersión para saber qué tan separados están.

Código

procesamiento.py

Python

```
frames = {
    1: [(10, 20), (30, 40), (50, 60), (70, 80), (90, 100), (110, 120)],
    2: [(15, 25), (35, 45)],
    3: [(100, 200), (120, 220), (140, 240),
        (160, 260), (180, 280), (200, 300),
        (220, 320), (240, 340), (260, 360),
        (280, 380), (300, 400), (320, 420),
        (340, 440), (360, 460), (380, 480),
        (400, 500)],
}

def obtener_zona(x):
    if x < 200:
        return "Carril 1"
    elif x < 400:
        return "Carril 2"
    else:
        return "Carril 3"

def calcular_densidad(n):
    if n <= 5:
        return "Baja"
    elif n <= 15:
        return "Media"
    else:
        return "Alta"

resultados = []

def procesar_frame(vehiculos):
    n = len(vehiculos)
    if n == 0:
        return None

    z1 = z2 = z3 = 0
```

```

for x, y in vehiculos:
    zona = obtener_zona(x)

    if zona == 1: z1 += 1
    elif zona == 2: z2 += 1
    else: z3 += 1

dispersion = 0
for i in range(n):
    for j in range(i+1, n):
        dx = vehiculos[i][0] - vehiculos[j][0]
        dy = vehiculos[i][1] - vehiculos[j][1]
        dispersion += (dx**2 + dy**2)

return {
    "n_vehiculos": n,
    "densidad": calcular_densidad(n),
    "zonas": {"Z1": z1, "Z2": z2, "Z3": z3},
    "dispersion": dispersion
}

```

Salida / Resultados

=== MÉTRICAS ===

Frame	Vehículos	Densidad	Z1	Z2	Z3	Disp
1	6	Media	0	0	6	1410341
2	2	Baja	0	0	2	48404
3	16	Alta	0	0	16	10851575

=== RENDIMIENTO ===

Vehículos	Tiempo (ms)
100	0.7492
500	23.0285
1000	94.5336
2000	381.8549

Actividad 2: Medir Tiempos de Ejecución

Se realizaron unas pruebas como las que hemos visto a lo largo del semestre para mostrar la eficiencia actual de esta versión del algoritmo con distintos tamaños de n que en este caso son los vehículos generados por coordenadas de datos aleatorios.

Código

test_procesamiento.py

Python

```
from procesamiento import procesar_frame
import random
import time
import matplotlib.pyplot as plt

def generar_frame(n):
    return [(random.randint(0,600), random.randint(0,400)) for _ in range(n)]

frames = {
    1: generar_frame(6),
    2: generar_frame(2),
    3: generar_frame(16),
}

print("\n=== MÉTRICAS ===\n")

print(f"{'Frame':<6} {'Vehículos':<10} {'Densidad':<10} {'Z1':<4} {'Z2':<4} {'Z3':<4} {'Disp':<10}")
print("-" * 60)

for f, vehiculos in frames.items():
    r = procesar_frame(vehiculos)
    z = r["zonas"]

    print(f"{f:<6} {r['n_vehiculos']:<10} {r['densidad']:<10} {z['Z1']:<4} {z['Z2']:<4} {z['Z3']:<4} {r['dispersion']:<10}")

# Medicion de tiempo
tamanos = [100, 500, 1000, 2000]
tiempos = []

for n in tamanos:
    total = 0

    for _ in range(10):
        frame = generar_frame(n)

        inicio = time.time()
        procesar_frame(frame)
        fin = time.time()

        total += (fin - inicio)

    promedio = (total / 10) * 1000
```

```
tiempos.append((n, promedio))

print("\n=== RENDIMIENTO ===\n")

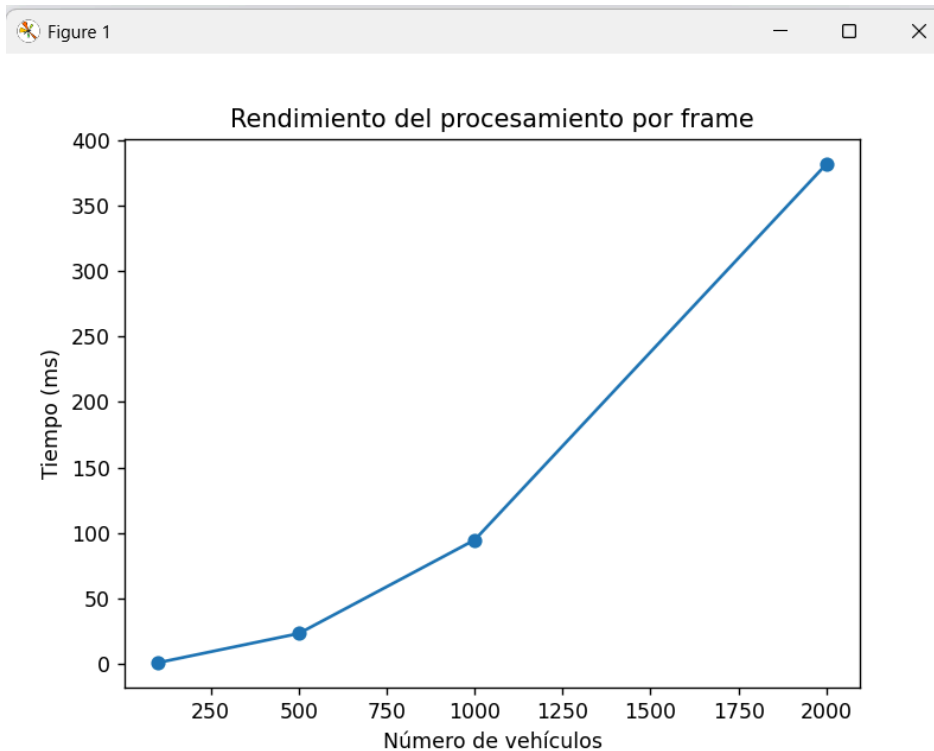
print(f"{'Vehículos':<12} {'Tiempo (ms)':<12}")
print("-" * 25)

for t in tiempos:
    print(f"{t[0]:<12} {t[1]:<12.4f}")

x = [t[0] for t in tiempos]
y = [t[1] for t in tiempos]

plt.plot(x, y, marker='o')
plt.xlabel("Número de vehículos")
plt.ylabel("Tiempo (ms)")
plt.title("Rendimiento del procesamiento por frame")
plt.show()
```

Gráfica de Rendimiento



Conclusión de Axel

Para este primer sprint todo salió tal cual lo planeado, las operaciones fueron simples y para posteriores sprint se tratará de hacer mejores métricas y escoger los mejores métodos para realizar la operación de procesar los frames

SPRINT 1 — Conclusión General

En este sprint el equipo logró familiarizarse con el contexto del problema de análisis de tráfico vehicular, así como con los módulos asignados a cada integrante. Se estableció el flujo general del sistema, identificando cómo se conectan las etapas desde la detección de vehículos, la generación y organización de datos, hasta su procesamiento para obtener métricas relevantes.

Además, se desarrollaron los primeros componentes funcionales del sistema, logrando detectar vehículos en video, generar datos estructurados por frame y procesarlos para obtener información como densidad y distribución del tráfico.

Limitaciones identificadas y próximos pasos:

Como limitaciones, se identificó que los módulos aún trabajan de forma independiente y no en un flujo completamente integrado en tiempo real. Asimismo, el análisis realizado es básico y no considera aún patrones complejos de comportamiento del tráfico.

Como siguientes pasos, se plantea integrar los módulos en un flujo continuo, trabajar con datos provenientes directamente de la detección real en video y ampliar el análisis mediante métricas más avanzadas, como patrones de tráfico y variaciones entre frames.